

✓ Agent-SimNet Policy Learning

Input Tensor Construction for a Simplified 8-GPU Topology

Feature Channels:

- **Channel 1 (L):** Propagation Delay matrix (Latency).
- **Channel 2 (B^{-1}):** Transmission Delay matrix (Inverse Bandwidth).

Modeling Assumptions:

- **Zero Self-Cost:** $L_{i,i} = 0$ and $B_{i,i}^{-1} = 0 \quad \forall i \in \{1, \dots, N\}$, where $N = 8$.
- **Transmission Delay:**
 - *Intra-node:* Lower, High bandwidth (e.g., NVLink).
 - *Inter-node:* Bigger, Low bandwidth (e.g., Ethernet).

```
import numpy as np

N = 8
# 2 channels: [Propagation Delay, Transmission Delay]
input_tensor = np.zeros((2, N, N))

# CHANNEL 1: PROPAGATION DELAY (just based on distance)
# I divide 8 GPUs into 2 nodes (4 GPUs each) to create a hierarchical (4-4) map
# Unit: Microseconds (us)
# Intra-node (same node): 0.1 us
# Inter-node (different nodes): 5.0 us

for i in range(N):
    for j in range(N):
        if i == j:
            input_tensor[0, i, j] = 0
        elif (i < 4 and j < 4) or (i >= 4 and j >= 4):
            input_tensor[0, i, j] = 0.1
        else:
            input_tensor[0, i, j] = 5.0

# CHANNEL 2: TRANSMISSION DELAY (capacity)(1 / Bandwidth)
# Unit: Seconds / Gigabytes
# I used a 4-4 split to model high-speed intra-node links (NVLink) vs. slower inter-node connections (Ethernet).
# NVLink Bandwidth: 300 GB/s -> Delay = 1/300 = 0.0033
# Ethernet Bandwidth: 10 GB/s -> Delay = 1/10 = 0.1

for i in range(N):
    for j in range(N):
        if i == j:
            input_tensor[1, i, j] = 0
        elif (i < 4 and j < 4) or (i >= 4 and j >= 4):
            input_tensor[1, i, j] = 0.0033 # faster(NVLink)
        else:
            input_tensor[1, i, j] = 0.1 # slower(Ethernet)

print("Input Tensor Shape:", input_tensor.shape)
print("-" * 50)
print("\nPropagation Delay Matrix:\n", input_tensor[0])
print("-" * 50)
print("\nPropagation Delay Matrix (First 4x4 - Intra-node):\n", input_tensor[0, :4, :4])
print("-" * 50)
print("\nPropagation Delay Matrix (Inter-node):")
print(input_tensor[0, :4, 4:])
print("-" * 50)
intra_trans = input_tensor[1, 0, 1]
inter_trans = input_tensor[1, 0, 7]
print(f"Transmission Delay:")
print(f"  > Intra-node (NVLink): {intra_trans:.4f}")
print(f"  > Inter-node (Ethernet): {inter_trans:.4f}")
```

Input Tensor Shape: (2, 8, 8)

Propagation Delay Matrix:

```
[[0.  0.1 0.1 0.1 5.  5.  5.  5. ]
 [0.1 0.  0.1 0.1 5.  5.  5.  5. ]
 [0.1 0.1 0.  0.1 5.  5.  5.  5. ]
 [0.1 0.1 0.1 0.  5.  5.  5.  5. ]
 [5.  5.  5.  5.  0.  0.1 0.1 0.1]
```

```
[5.  5.  5.  5.  0.1 0.  0.1 0.1]
[5.  5.  5.  5.  0.1 0.1 0.  0.1]
[5.  5.  5.  5.  0.1 0.1 0.1 0.  ]]
-----
```

Propagation Delay Matrix (First 4x4 - Intra-node):

```
[[0.  0.1 0.1 0.1]
 [0.1 0.  0.1 0.1]
 [0.1 0.1 0.  0.1]
 [0.1 0.1 0.1 0.  ]]
```

Propagation Delay Matrix (Inter-node):

```
[[5.  5.  5.  5.]
 [5.  5.  5.  5.]
 [5.  5.  5.  5.]
 [5.  5.  5.  5.]]
```

Transmission Delay:

```
> Intra-node (NVLink): 0.0033
> Inter-node (Ethernet): 0.1000
```

Agent Neural Network

-Simple **3-layer MLP with 2 hidden layers architecture based Agent** to map the topology tensor to a link-level policy matrix

Preprocessing

- The input tensor is flattened from $2 \times 8 \times 8$ to a **128-dimensional vector**
- BatchSize:1 (one topology)

Architecture

- Flatten $\rightarrow$ Linear(128, 128) + ReLU
- Linear(128, 64) + ReLU
- Linear(64, 64) + Sigmoid
- Reshape to 8×8

Output

- Policy matrix: $P \in [0, 1]^{8 \times 8}$
- Each entry $P_{i,j}$ gives a **soft score** for selecting the directed link from GPU i to GPU j

```
input_tensor_torch = torch.from_numpy(input_tensor).float()

# Agent architecture (The Student Model)
class CommunicationAgent(nn.Module):
    def __init__(self, N=8):
        super(CommunicationAgent, self).__init__()
        self.N = N

        # Flatten: 2 * 8 * 8 = 128
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(2 * N * N, 128)
        self.fc2 = nn.Linear(128, 64)

        #Output
        self.output_layer = nn.Linear(64, N * N)

    def forward(self, x):
        # x shape: (Batch, 2, 8, 8)
        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))

        # Score for each link
        policy_scores = torch.sigmoid(self.output_layer(x))

        # (Batch_size, N, N)
        return policy_scores.view(-1, self.N, self.N)

agent = CommunicationAgent(N=8)
policy_tensor = agent(input_tensor_torch.unsqueeze(0))

print("--- AGENT OUTPUT ---")
print(f"Input Shape: {input_tensor_torch.shape}")
print(f"Policy Tensor Shape: {policy_tensor.shape}")

print(f"\nMin score: {policy_tensor.min().item():.4f}")
print(f"Max score: {policy_tensor.max().item():.4f}")
print(f"Mean score: {policy_tensor.mean().item():.4f}")
#everything is around 0.5- not trained yet
```

```

print("\nFull Policy Matrix:")
print(policy_tensor[0].detach().numpy())

print("\nDiagonal scores (self-links):")
print(torch.diag(policy_tensor[0]).detach().numpy())

--- AGENT OUTPUT ---
Input Shape: torch.Size([2, 8, 8])
Policy Tensor Shape: torch.Size([1, 8, 8])

Min score: 0.3205
Max score: 0.6926
Mean score: 0.4962

Full Policy Matrix:
[[0.45887807 0.5156146 0.35126045 0.5580972 0.3346188 0.4793611
 0.44692025 0.58806497]
 [0.5306832 0.6135461 0.32046452 0.5567273 0.44717363 0.46936658
 0.5295387 0.48856753]
 [0.48060894 0.47747862 0.60186064 0.46306226 0.51691926 0.34052697
 0.4342628 0.47701395]
 [0.36563748 0.4939649 0.46302125 0.4719045 0.4619942 0.5191521
 0.38324118 0.40849817]
 [0.3777215 0.6925742 0.5301102 0.6153601 0.36661804 0.4685325
 0.5071671 0.42487448]
 [0.48429582 0.42377794 0.5569597 0.4088371 0.61364377 0.4914498
 0.5276565 0.63195914]
 [0.5363204 0.45247212 0.40283394 0.6894477 0.6236773 0.620894
 0.5882942 0.5570513 ]
 [0.5243092 0.6008833 0.46098822 0.5262352 0.48530054 0.54991806
 0.40281463 0.56782067]]

Diagonal scores (self-links):
[0.45887807 0.6135461 0.60186064 0.4719045 0.36661804 0.4914498
 0.5882942 0.56782067]

```

SimNet (Proxy Model)

Goal: estimate a scalar completion time from the topology and the Agent output(Policy Score Matrix).

- **Input 1:** topology tensor, shape $(2 \times 8 \times 8)$
- **Input 2:** policy matrix from the Agent, shape (8×8)
- **Fusion:** topology (2 channels) + policy (1 channel) $\rightarrow$ 3-channel input (concatting)
- **Architecture:** Flatten $\rightarrow 192 \rightarrow 128 \rightarrow 64 \rightarrow 1$
- **Output:** a single non-negative scalar interpreted as an estimated completion value

At this stage, SimNet does **not** compute the real communication time analytically.

It only produces a learned scalar prediction from the topology-policy pair.

Since the model is not trained yet, the current output is only a pipeline check.

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class SimNet(nn.Module):
    def __init__(self, N=8):
        super().__init__()
        self.N = N
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(3 * N * N, 128)
        self.fc2 = nn.Linear(128, 64)
        self.output_layer = nn.Linear(64, 1)

    def forward(self, topology, policy):
        # topology: (Batch, 2, N, N)
        # policy: (Batch, N, N) or (Batch, 1, N, N)

        if policy.dim() == 3:
            policy = policy.unsqueeze(1) # (Batch, 1, N, N)

        x = torch.cat((topology, policy), dim=1) # (Batch, 3, N, N)

        x = self.flatten(x) # (Batch, 3*N*N)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))

        completion_time = F.relu(self.output_layer(x)) # non-negative scalar
        return completion_time

simnet = SimNet(N=8)

estimated_time = simnet(

```

```

        input_tensor_torch.unsqueeze(0), # topology: (1, 2, 8, 8)
        policy_tensor                    # policy:   (1, 8, 8)
    )

    print("--- SIMNET (PROXY MODEL) ---")
    print(f"Input channels: Topology (2) + Policy (1) = 3")
    print(f"Estimated completion value: {estimated_time.item():.6f}")

--- SIMNET (PROXY MODEL) ---
Input channels: Topology (2) + Policy (1) = 3
Estimated completion value: 0.154203

```

Heuristic Policy Solution:

- **Self-links ignored**

$$P_{i,i}^{(h)} = 0 \quad \forall i = 1, \dots, N$$

- **Intra-node links are preferred**

$$P_{i,j}^{(h)} = 1.0 \quad \text{if } i \text{ and } j \text{ are in the same node}$$

- **Inter-node links are penalized**

$$P_{i,j}^{(h)} = 0.1 \quad \text{if } i \text{ and } j \text{ are in different nodes}$$

This defines a simple rule-based baseline policy consistent with the assumed hierarchical topology.

```

def create_heuristic_policy(N=8):
    teacher_policy = np.zeros((N, N))
    for i in range(N):
        for j in range(N):
            if i == j:
                teacher_policy[i, j] = 0
            elif (i < 4 and j < 4) or (i >= 4 and j >= 4):
                teacher_policy[i, j] = 1.0 #intra-node
            else:
                teacher_policy[i, j] = 0.1 #inter-node
    return torch.from_numpy(teacher_policy).float()
ideal_policy = create_heuristic_policy(N=8)

```

Agent Training (Heuristic Imitation)

The agent is trained using supervised learning to approximate the heuristic policy.

- **Input:** topology tensor T
- **Target:** heuristic policy $P^{(h)}$
- **Prediction:** $P_{\text{agent}} = f_{\theta}(T)$

Loss Function:

Mean Squared Error (MSE) is used to measure the difference between predicted and heuristic policies:

$$L = \|P_{\text{agent}} - P^{(h)}\|^2$$

Training Procedure:

- 3 Layer MLP
- 100 epochs using the Adam optimizer with a learning rate of 0.01.
- MSE loss between prediction and heuristic policy
- Update parameters using Adam optimizer

```

#Training
optimizer = torch.optim.Adam(agent.parameters(), lr=0.01)
criterion = nn.MSELoss()
for epoch in range(100):
    optimizer.zero_grad()

    # Agent's prediction
    student_policy = agent(input_tensor_torch.unsqueeze(0))

    #Loss
    # student_policy: (1, 8, 8), ideal_policy: (8, 8)
    # unsqueeze here used for dims to match
    loss = criterion(student_policy, ideal_policy.unsqueeze(0))

    # update
    loss.backward()
    optimizer.step()

```

```
if (epoch + 1) % 20 == 0:
    print(f"Epoch [{epoch+1}/100], Loss: {loss.item():.6f}")
```

```
Epoch [20/100], Loss: 0.005000
Epoch [40/100], Loss: 0.005000
Epoch [60/100], Loss: 0.005000
Epoch [80/100], Loss: 0.005000
Epoch [100/100], Loss: 0.005000
```

```
trained_policy = agent(input_tensor_torch.unsqueeze(0))
diff = (trained_policy[0] - ideal_policy).abs()
print("\nMean Absolute Error:", diff.mean().item())
```

```
Mean Absolute Error: 0.05000000447034836
```

Performance Evaluation: Simulated Completion Time

I approximate the total communication time as a weighted sum:

$$T(P) = \sum_{i,j} C_{i,j} \cdot P_{i,j} \quad \text{where} \quad C_{i,j} = L_{i,j} + B_{i,j}^{-1}$$

This represents a cost-weighted communication model, where:

- high-cost links should receive lower weights
- low-cost links should be prioritized

Goal: learn a policy P that minimizes total communication cost.

$$P^* = \arg \min_P T(P)$$

This allows us to quantitatively compare different policies (heuristic vs learned).

```
def calculate_real_completion_time(topology, policy):
    # prop_delay (Channel 1): Physical distance/latency between GPUs
    # trans_delay (Channel 2): Time to push data bits (Inverse Bandwidth)
    prop_delay = topology[0, 0]
    trans_delay = topology[0, 1]
    # Calculating Total Communication Cost
    # I computed the element-wise weighted sum of the costs and the Agent's policy.
    # This represents the total time required for the Allgather operation.
    total_time = torch.sum(prop_delay * policy + trans_delay * policy)

    # Normalization
    return total_time / 100.0 # feature scaling technique ensuring that the output values are small enough to prevent 'Exp
```

```
#Calibrating the enviroment-- Adam optimizer to fine-tune SimNet's weights
sim_optimizer = torch.optim.Adam(simnet.parameters(), lr=0.005)

for i in range(200):
    sim_optimizer.zero_grad()

    # The Agent generates a policy P based on the current topology
    policy = agent(input_tensor_torch.unsqueeze(0))

    # SimNet predicts a completion time for this specific policy
    predicted_time = simnet(input_tensor_torch.unsqueeze(0), policy)

    # The actual completion time computed by analytical formula
    real_time = calculate_real_completion_time(input_tensor_torch, policy)

    # Minimize the MSE between the prediction and real value
    loss = criterion(predicted_time, real_time.reshape(1, 1))
    loss.backward()
    sim_optimizer.step()
print(f"Final Alignment Error: {loss.item():.10f}")
```

```
Final Alignment Error: 0.0000000000
```

```
# Test Topology
test_input = np.zeros((2, N, N))

for i in range(N):
    for j in range(N):
        if i == j: continue
        #again 2 nodes as 4-4
        if (i < 4 and j < 4) or (i >= 4 and j >= 4):
            test_input[0, i, j] = 0.1
            test_input[1, i, j] = 0.003
```

```

        else:
            # Inter-node - more delay
            test_input[0, i, j] = 20.0
            test_input[1, i, j] = 0.5

test_tensor = torch.from_numpy(test_input).float().unsqueeze(0)
# Agent
with torch.no_grad():
    test_policy = agent(test_tensor)

# SimNet
with torch.no_grad():
    predicted_time = simnet(test_tensor, test_policy)
print(f"Predicted Time: {predicted_time.item():.4f} ms")

print(f"\nNode 1 -> Node 2 Policy Score (0->5): {test_policy[0, 0, 5].item():.6f}")
print(f"Node 1 -> Node 1 Policy Score (0->1): {test_policy[0, 0, 1].item():.6f}")

```

Predicted Time: 4.4640 ms

Node 1 -> Node 2 Policy Score (0->5): 0.000000

Node 1 -> Node 1 Policy Score (0->1): 1.000000

```

import matplotlib.pyplot as plt
import seaborn as sns

def plot_policy(policy_tensor, title="Agent Policy Matrix"):
    plt.figure(figsize=(10, 8))
    data = policy_tensor[0].cpu().detach().numpy()

    sns.heatmap(data,
                annot=True,
                fmt=".2f",
                cmap="YlGnBu",
                cbar_kws={'label': 'Connection Score (0-1)'})

    plt.title(title)
    plt.xlabel("Target GPU ID (Receiver)")
    plt.ylabel("Source GPU ID (Sender)")

    plt.xticks(range(8))
    plt.yticks(range(8))

    plt.show()

plot_policy(test_policy, title="Final Test: Optimized Policy")

```

